

Interactive Air Hockey Robot: Decentralized PID Control for Real-Time Puck Interception

Robotics Team 4: “What The Puck”

Hannah Yared, Spencer Kelly, John Meshinsky, Morganne Malloy, Tony Zong
Department of Mechanical and Aerospace Engineering
University of California, Los Angeles

Abstract—This project developed a vision-based robotic air hockey system for real-time puck interception. The robot uses a 2-DOF belt-driven mechanism, Dynamixel servo motors, an overhead Raspberry Pi camera, and a custom paddle. HSV color thresholding detects the puck and paddle in image coordinates. The system estimates the puck’s linear trajectory from the puck and paddle positions, then moves the end effector to the predicted intercept position. A decentralized PID controller reduces error between the predicted end effector target and measured end effector position. Hardware puck-launch tests evaluated RMS error, peak error, settling time, and overshoot. The final controller achieved an RMS error of approximately 1.54 cm, peak error of approximately 6.94 cm, settling time of approximately 0.85 s, and overshoot of approximately 0.8 cm during slow-to-moderate puck trials. The system demonstrated real-time puck detection, trajectory prediction, and defensive blocking. Performance was limited by camera frame rate, communication latency, mechanical backlash, and belt tension near workspace limits.

Index Terms—Robotics, Feedback Control, PID Control, Feedforward Control, Trajectory Tracking, Machine Vision, Air Hockey Robot, Real-Time Control

I. INTRODUCTION

A. Project Motivation

This project aims to develop a 2-DOF robotic air hockey system capable of detecting, tracking, predicting, and intercepting a moving puck in real time. Many classic games are multiplayer and require more than one person to actively participate in the game. However, if one was interested in playing the game alone, a human-versus-robot gaming system could be implemented. This concept can be applied to a variety of other systems such as sports training/rehabilitation robotics, and human-robot collaboration systems that require a rapid response to human actions. Robotic interception is important as it requires simultaneous perception, prediction, and control, and is key for robotic sorting, tracking, and defense systems.

B. Literature Review

A significant challenge in multi-degree-of-freedom manipulators is the dynamic coupling that exists between joints. Traditional model-based control approaches often require analysis of the complete coupled system, resulting in increased computational complexity. To address this issue, Sun et al. proposed a decentralized robust exact tracking controller for

a 2-DOF planar robot manipulator that treats coupling effects and model uncertainties as disturbances and compensates for them locally at each joint [1]. Through disturbance estimation and compensation, the manipulator joints become effectively decoupled, allowing each joint to be analyzed as an independent single-input single-output system.

The primary contribution of the work by Sun et al. is the demonstration that Lyapunov stability of each individual joint guarantees stability of the overall manipulator when the decentralized control architecture is employed [1]. This significantly simplifies controller design compared to conventional centralized approaches, which require stability analysis of the full nonlinear robotic system. Furthermore, the decentralized architecture reduces computational requirements while maintaining robust trajectory tracking performance in the presence of uncertainties and external disturbances.

The control strategy presented in [1] is particularly relevant to this robotic air hockey system. Like the 2-DOF planar manipulator studied by Sun et al., the present system consists of two orthogonal prismatic axes with relatively low inertial coupling, allowing each axis to be controlled independently. Both systems treat residual coupling effects as disturbances to be rejected by local feedback loops rather than modeled explicitly, which simplifies implementation and is well-suited to real-time applications with tight computational budgets. A key difference, however, is that Sun et al. derived a model-based disturbance estimator to achieve robust exact tracking, whereas this project employs empirically tuned PID feedback operating in image (pixel) space rather than joint space. This trade-off sacrifices some theoretical robustness guarantees in exchange for implementation simplicity and natural alignment with the vision-based sensing pipeline. Both approaches, nonetheless, converge on the conclusion that decentralized control is a viable and computationally efficient solution for multi-axis manipulator systems with weak inter-axis coupling.

C. Project Objectives

The objective of this project is to develop a 2-DOF robotic air hockey system capable of autonomously detecting, tracking, predicting, and intercepting a moving puck in real time through the integration of machine vision and decentralized feedback control. The following points are key project goals:

- 1) **Real-Time Puck Detection and Tracking:** Develop a vision-based tracking system capable of detecting the puck and paddle positions throughout the workspace in real time.
- 2) **Accurate end effector Position Control:** Design and implement a decentralized controller for the two prismatic axes to achieve accurate end effector positioning and trajectory tracking.
- 3) **Autonomous Puck Interception:** Integrate the vision and control subsystems to predict and intercept incoming puck trajectories in real time.

System performance is evaluated using metrics including puck tracking accuracy, end effector positioning error, and overall delay in convergence. The system is considered successful if it maintains stable closed-loop control and consistently tracks the prediction of incoming pucks within the robot's reachable workspace.

II. SYSTEM OVERVIEW

A. Robot Description

The robotic air hockey system is modeled after an open-source robot design [2], [3]. It consists of a two-degree-of-freedom (2-DOF) planar manipulator designed to position an end effector within the playing area rapidly. Motion is generated through two orthogonal linear axes driven by Dynamixel servo motors and a timing belt with a 4:1 gear ratio mounted on linear rails. By commanding both motors in the same direction, the end effector translates horizontally across the table. Commanding the motors in opposite directions produces vertical motion. Simultaneous actuation of both motors at different speeds allows diagonal trajectories, enabling the robot to intercept pucks approaching from arbitrary directions.

Each Dynamixel servo motor is responsible for regulating motion along one degree of freedom. The decentralized control architecture treats each axis independently, simplifying controller design and reducing computational complexity while maintaining accurate position tracking.

Puck detection is performed using a Raspberry Pi Camera Module capable of capturing frames at 80 FPS via the Picamera2 interface. Each frame is converted to the HSV color space and color-thresholded to isolate the puck, with the centroid extracted from the largest detected contour. The puck's linear trajectory is estimated from the detected puck and human-controlled paddle positions, then projected forward geometrically to predict an intercept point along the robot's defender line. The predicted intercept coordinate is mapped to robot task-space before being issued as a motor setpoint.

The end effector consists of a custom 3D-printed defender designed to interact with the air hockey puck. The defender is rigidly attached to the moving carriage and serves as the contact interface between the robot and the puck. Together, the vision system, decentralized controller, and 2-DOF manipulator form an integrated robotic interception platform capable of autonomous air hockey gameplay.

B. Hardware Components

- Two Dynamixel servo motors
- Raspberry Pi 4B + Camera Module V2
- U2D2 interface
- 12 V 10 A power supply and 12 V DC adapter plug
- 2.5 m GT2 6 mm timing belt
- 4×8 mm×300 mm linear rods
- 4×8 mm linear motion bearings, M3 nuts and bolts
- Air hockey table and puck

C. System Architecture

The system architecture of the autonomous air hockey robot senses the puck and the paddle of the opponent to feed into the kinematics to command the motors to move the end effector paddle. The position data of the end effector is fed into the loop to correct its position.

The vision system uses a Raspberry Pi camera to capture the optical data into an RGB grid. This feeds into the software, where it will detect a cluster of data with similar and easily detectable RGB values in a mask to highlight the three key objects' positions in the task space: the paddle (green), puck (red), and the end effector (blue). This data feeds into the trajectory prediction software.

Trajectory prediction places the three objects into the task space relative to their positions in the camera grid, bound by the bounds of the table. The linear slope between the paddle and puck intercepting the defending line is the desired positional location data of the end effector. This data, as well as the positional data of the end effector, informs the controller.

The controller computes the positional data error and adjusts the data to determine the Cartesian position commands to convert into the inverse kinematics to command the motors to operate the belt system to move the end effector to the desired position. Next, a continuous loop in the controller is fed the error of the desired and actual position of the end effectors to compute the commands to make up the error, so the end effector moves to the desired positions.

D. Robot Visualization

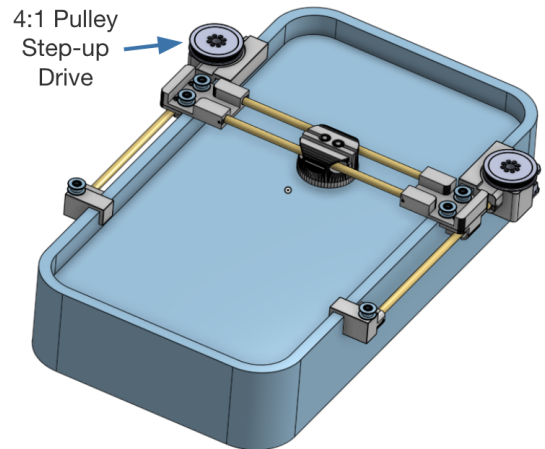


Fig. 1: Interactive air hockey robot CAD model

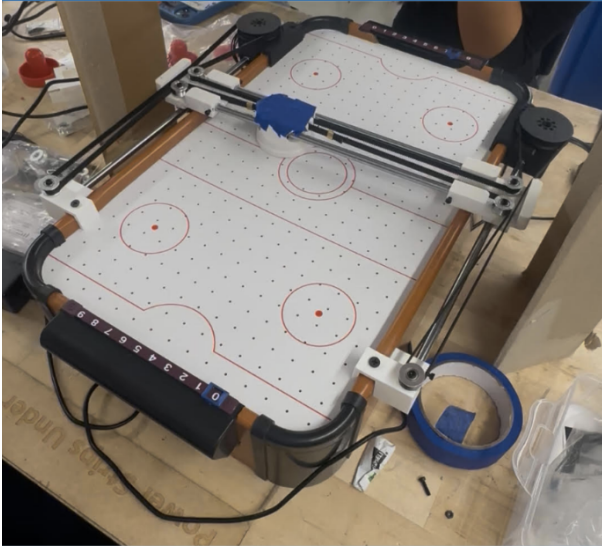


Fig. 2: Physical air hockey robot with overhead Raspberry Pi camera (not shown) mounted above the table. The green opponent paddle, red puck, and blue end effector paddle are visible and tracked via HSV color thresholding.

III. METHODS

A. Control Design Requirements

The control system must satisfy four core performance requirements driven by the physical constraints of real-time puck interception and prediction.

- 1) **Tracking Error:** The end effector must reach the predicted intercept position with sufficient accuracy to make contact with the puck. A deadband of ± 5 px (~ 0.2 cm) was enforced in software, below which no corrective command is issued. Steady-state error beyond this threshold results in a missed interception.
- 2) **Response Time and Settling Time:** The puck crosses the table in a fraction of a second, so the end effector must begin moving immediately when a new intercept is predicted and settle before the puck arrives. The controller time step is capped at $\Delta t = 0.10$ s to prevent large velocity spikes from stale frames, and motor profile velocity is set to a addressed magnitude of 230 (55.67 rev/min) to allow the arm to travel up to ~ 13.7 cm/s at full command.
- 3) **Accuracy and Repeatability:** The task space spans ± 9.6 cm horizontally and -11.2 to $+8.75$ cm vertically, with 23 deg/cm motor scaling on both axes. The controller must position the end effector repeatably across this workspace, including near the edges where belt tension increases and mechanical resistance is highest.
- 4) **Stability:** The integral term is clamped to ± 200 px to prevent windup during periods of large or sustained error. Motor commands are saturated at ± 600 cm/s to keep the arm within safe operating limits and avoid over-torque events that could damage the belt drive.

B. Modeling Assumptions

To simplify the controller design, the following assumptions were made:

- **Motion in Free Space:** The robot operates without physical contact with the environment during trajectory tracking. Interaction forces are only present during brief puck impacts and are neglected in the controller design.
- **Joint Space Regulation:** Control is performed independently in joint space, where each motor is commanded directly to achieve the desired position.
- **Low Inertial Coupling:** Motion along the horizontal axis has a negligible effect on the dynamics of the vertical axis, and vice versa.
- **Decentralized Dynamics:** The system is modeled as two independent single-input single-output (SISO) control loops, one for each motor.
- **Coupling as Disturbance:** Residual nonlinearities, unmodeled dynamics, friction, and axis coupling effects are treated as external disturbances and compensated through feedback control.
- **Rigid Body Assumption:** Structural flexibilities, belt compliance, and end effector deformation are assumed to be negligible.
- **Planar Motion:** Motion is constrained to the plane of the air hockey table, resulting in a two-degree-of-freedom system.

C. System Modeling

The air hockey robot is modeled for precision positioning and puck trajectory prediction to overcome the inherent delay of the vision system. The 2-DOF design was intended to support both defensive and offensive operation within the task space; however, this project focuses on the defensive (blocking) mode, in which control is exercised along a single lateral axis.

The assumptions made for modeling the system accounted for the higher frequency and lower speed of the motors, the limited resolution and refresh rate of the Raspberry Pi camera, the rigidity of the linkage, and the elastic and linear motion of the puck. Although the servos were able to operate at a higher frequency, they were limited to the commands and directions of the camera, which often ran at a refresh rate of 5 to 15 frames per second. To overcome this bottleneck, linear motion of the puck is assumed. Predictions of the puck's motion could be made by measuring the slope between the paddle and the puck, creating a predicted trajectory that the end effector could meet to defend against it. The system also assumes negligible operational vibrations.

The plant kinematics operate through the simultaneous rotation of both actuators. The servos operate in extended position mode to move the end effector. When the motors rotate symmetrically with each other, the end effector translates laterally (along the y -axis); commanding them in opposite directions produces longitudinal motion (along the x -axis). For the defensive blocking task, the servos were fixed to move synchronously to produce pure lateral displacement.

The joint rotation of the motors was mapped to Cartesian task-space coordinates using calibrated scaling factors of 23 degrees of rotation per centimeter, expressed as:

$$\theta_A = \theta_{A,\text{home}} + \text{SIGN}_X \cdot K_x \cdot x + \text{SIGN}_Y \cdot K_y \cdot y_{\text{fixed}} \quad (1)$$

$$\theta_B = \theta_{B,\text{home}} + \text{SIGN}_X \cdot K_x \cdot x - \text{SIGN}_Y \cdot K_y \cdot y_{\text{fixed}} \quad (2)$$

$$\dot{\theta}_A = K_x \cdot \dot{x} = K_x \cdot v_x \quad (3)$$

$$\dot{\theta}_B = K_x \cdot \dot{x} = K_x \cdot v_x \quad (4)$$

where $K_x = K_y = 23 \text{ deg/cm}$ and SIGN_X , SIGN_Y are direction factors (both positive for pure y -axis lateral motion). In the continuous s -domain, the transfer function $G_p(s)$ mapping the commanded task-space velocity $V_x(s)$ to the true physical position $X(s)$ simplifies to a pure integrator with an unmodeled high-frequency servo time constant τ_m :

$$G_p(s) = \frac{X(s)}{V_x(s)} \approx \frac{1}{s(\tau_m s + 1)} \quad (5)$$

The Raspberry Pi Camera operated at 15 Hz under full system load, introducing transport delay that must be compensated by the controller. Because the camera grid and task space are related by a fixed pixel-to-centimeter scale, all PID control is performed in pixel space and converted to motor commands only at the output stage.

D. Trajectory Prediction and Tracking

The camera captures frames at a maximum of 80 FPS at a resolution of 640×480 pixels. Each frame is processed to locate the puck and the opponent's paddle using color thresholding in the HSV color space, which is more robust to lighting variation than RGB. Before thresholding, the frame is lightly blurred to reduce noise from reflections off the puck surface. Pixels within a tuned color range are kept, and small isolated blobs are removed through a brief erosion and dilation step. The center of the largest remaining region is taken as the object's position:

$$\mathbf{p} = \left(\frac{m_{10}}{m_{00}}, \frac{m_{01}}{m_{00}} \right) \quad (6)$$

where m_{ij} are standard image moments. This gives pixel coordinates $\mathbf{p}_{\text{puck}} = (p_x, p_y)$ and $\mathbf{p}_{\text{paddle}} = (d_x, d_y)$ each frame that both objects are visible.

A notable feature of this trajectory prediction strategy is that the line of action of the puck is inferred from the relative positions of the opponent's paddle and the puck, even when the puck is stationary. This allows the system to anticipate intent based on the opponent's body positioning, analogous to how a human goalkeeper reads a penalty kick before the ball is struck. The puck's direction of travel is estimated as:

$$(\Delta x, \Delta y) = (p_x - d_x, p_y - d_y) \quad (7)$$

This direction is stepped forward, segment by segment, until the puck's projected path reaches the defender line at $x_{\text{def}} = 455 \text{ px}$. At each step, the algorithm checks whether the puck

will hit a wall before reaching the defender line by comparing two travel distances: the distance to the defender line and the distance to the nearest wall.

If the puck prediction hits a wall first, the segment is advanced to the wall contact point, the vertical direction is flipped ($\Delta y \leftarrow -\Delta y$), and the process repeats for up to six bounces. If the puck is not moving toward the defender ($\Delta x = 0$), no prediction is made and no motor command is issued that frame. The code then runs functions for intercept clamping and target smoothing. The predicted intercept may land outside the physical goal opening, so it is clipped to the reachable range $[y_{\text{top}}^{\text{goal}}, y_{\text{bot}}^{\text{goal}}]$:

$$y_{\text{clamped}} = \text{clip}(y_{\text{pred}}, y_{\text{top}}^{\text{goal}}, y_{\text{bot}}^{\text{goal}}) \quad (8)$$

with $y_{\text{top}}^{\text{goal}} = 120 \text{ px}$ and $y_{\text{bot}}^{\text{goal}} = 340 \text{ px}$. Since the predicted position can jump frame-to-frame on a fast-moving puck, a lightweight exponential moving average (EMA) is applied to smooth the target before it is sent to the controller:

$$\hat{y}_k = \alpha y_{\text{clamped},k} + (1 - \alpha) \hat{y}_{k-1}, \quad \alpha = 0.65 \quad (9)$$

The filter is reset whenever the puck or paddle is lost from view, so that an old prediction does not linger when tracking resumes. The final target seen by the controller each frame is:

$$\mathbf{target} = (x_{\text{def}}, \hat{y}_k). \quad (10)$$

E. Controller Selection

A decentralized control architecture was selected because it provides a computationally efficient solution for real-time air hockey while avoiding the complexity of model-based approaches such as inverse dynamics control [1]. Since the robot operates on two linear axes with minimal dynamic coupling, independent control of each motor was sufficient to achieve the desired positioning performance.

PID control was chosen due to its simplicity, robustness, and effectiveness for position regulation. The proportional term improves responsiveness to position errors, the integral term eliminates steady-state error, and the derivative term helps reduce overshoot and oscillations. Furthermore, the Dynamixel actuators used in this project contain built-in PID position control capabilities, simplifying implementation and tuning.

Rather than performing error correction directly in joint space, the primary feedback loop occurs through the vision system. The camera continuously measures the puck's center of mass in image coordinates and compares its observed position to the predicted interception location. The resulting pixel error is converted into a desired paddle position and transmitted to the motor controllers.

F. Controller Architecture

The main control loop for the robot utilizes a Decentralized PID controller for the task space that processes the positional error from the vision space. The system uses feedback and feedforward terms to maintain the stability of the system, which relies on predictive positioning.

The feedback error term is defined as:

$$e_x[k] = \hat{Y}_{\text{tgt}}[k] - Y_{\text{ee}}[k] \quad (11)$$

where $\hat{Y}_{\text{tgt}}[k]$ is the EMA-smoothed predicted intercept position (in pixels) and $Y_{\text{ee}}[k]$ is the measured end effector position (in pixels) at time step k . The discrete PID control law then produces a commanded velocity:

$$v_x[k] = K_P e_x[k] + K_I \sum_{j=0}^k e_x[j] \Delta t + K_D \frac{e_x[k] - e_x[k-1]}{\Delta t} \quad (12)$$

The proportional (K_P), integral (K_I), and derivative (K_D) gains use this positional error in the pixel grid from the desired position of the predicted trajectory intercept to correct for transient errors. This rejects physical disturbances and eliminates steady-state positioning drift along the defensive axis.

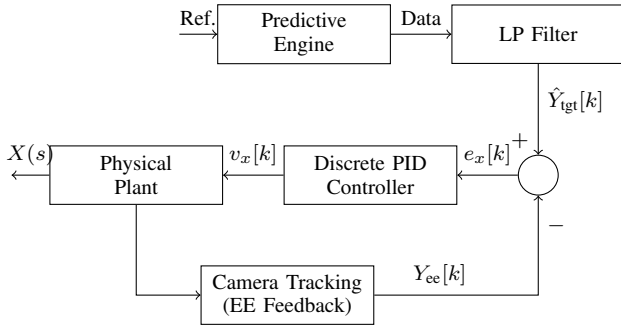


Fig. 3: Block diagram of the closed-loop visual-servo control architecture. The Predictive Engine estimates the puck intercept point from the paddle and puck pixel positions. An exponential moving average (LP Filter) smooths the target $\hat{Y}_{\text{tgt}}[k]$ before it is compared with the measured end effector position $Y_{\text{ee}}[k]$ (obtained via camera tracking). The resulting error $e_x[k]$ drives the discrete PID controller, which outputs a velocity command $v_x[k]$ to the physical plant (Dynamixel motors and belt drive). The loop closes through the camera, which measures the updated end effector position each frame.

A feedforward term based on the instantaneous puck velocity was planned to minimize phase lag; however, the low and variable camera frame rate made reliable velocity estimates infeasible. The intended wall-bounce velocity reversal was given by:

$$v_{y,\text{post}} = -v_{y,\text{pre}} \quad (13)$$

This term was therefore omitted from the final implementation, and all tracking performance relies on PID feedback alone.

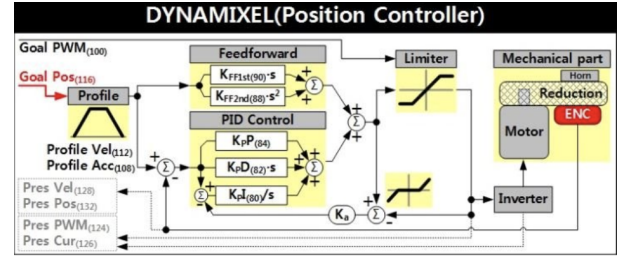


Fig. 4: Full system architecture diagram illustrating the signal flow from the PiCamera sensor through the vision processing pipeline, bounce prediction and smoothing, visual-servo PID controller, motor kinematics, and Dynamixel actuators. The telemetry logger records time, end effector position, target position, error, and gains to a CSV file for offline analysis.

G. Gain Selection and Tuning

The tuning of the decentralized task-space PID controller was performed iteratively using telemetry logs to record and compare the rate of convergence and the magnitude of overshoot. The objective was to achieve a stiff, critically damped response that immediately reacts to updated prediction data.

The proportional gain K_P was adjusted first, starting at a value of 10 and stepping down logarithmically through 1 and 0.1, eventually settling in the range $[0.3, 0.4]$. Too low a value caused the end effector to lag far behind the target; too high a value caused oscillation. Once a suitable K_P range was established, K_I and K_D were tuned sequentially. The derivative gain K_D was introduced to dampen oscillation and counteract the phase lag introduced by the low-frame-rate vision loop. The integral gain K_I was kept small to eliminate minor steady-state offsets due to joint friction and rail tolerance without causing windup or boundary overflow. The final experimentally tuned gains are listed in Table I.

Controller Parameter	Symbol	Final Value
Proportional Gain	K_P	0.3
Integral Gain	K_I	0.004
Derivative Gain	K_D	0.05

TABLE I: Experimentally tuned task-space PID controller gains.

IV. EXPERIMENTAL PROCEDURE

A. Hardware Testing

Hardware testing was performed on the physical air hockey robot using an overhead camera, Dynamixel motors, belt-driven axes, and a custom end effector. The table was placed flat on the ground. The camera was mounted at a fixed height above the table and remained stationary for all trials.

HSV thresholds were tuned to isolate the puck, human-controlled paddle, and end effector from the table background. The center of each object was calculated from its contour centroid in pixel coordinates. The detected puck and human-controlled paddle positions were used to estimate the puck's

linear trajectory. The measured end effector position was used as feedback for control. The robot was also tested without puck motion to verify motor direction, workspace limits, and camera feedback.

Puck-launch trials were performed from the human side of the table. For each trial, the predicted puck trajectory determined the desired end effector position. The PID controller adjusted motor commands to reduce the error between the desired and measured end effector positions. PID gains were tuned by observing response speed, overshoot, oscillation, and steady-state error. Proportional gain was increased to improve response speed, derivative gain was adjusted to reduce overshoot, and integral gain was limited to reduce drift and windup.

To assess controller gain sensitivity, two gain configurations were compared under otherwise identical conditions: the final tuned gains ($K_P = 0.4$, $K_I = 0.009$, $K_D = 0.0325$) and a slightly reduced configuration ($K_P = 0.3$, $K_I = 0.004$, $K_D = 0.05$). Each configuration was evaluated over repeated slow-to-moderate puck launches from a consistent launch position to reduce human variability in test conditions.

B. Performance Metrics

The primary controller error was the end effector position error,

$$e_x(t) = x_{ee,d}(t) - x_{ee}(t), \quad (14)$$

where $x_{ee,d}(t)$ is the desired end effector position from the predicted puck trajectory, and $x_{ee}(t)$ is the measured end effector position. RMS error was used to quantify average end effector tracking error:

$$e_{RMS} = \sqrt{\frac{1}{N} \sum_{i=1}^N e_x(i)^2}. \quad (15)$$

Peak error was defined as the maximum end effector position error:

$$e_{peak} = \max e_x(t). \quad (16)$$

Settling time was defined as the time required for the end effector error to remain within a ± 5 px (~ 0.2 cm) band around zero. Overshoot was defined as the maximum distance the end effector moved past the desired position before returning toward the target.

V. RESULTS

A. Controller Demonstration

The robot was successfully demonstrated performing real-time defensive blocking against human opponents. The system detected the puck and opponent paddle positions, predicted the puck's trajectory to the defender line, and commanded the end effector to the predicted intercept position. Video captures of the robot in operation are included in the supplemental materials (see Appendix F).

B. Trajectory Tracking Performance

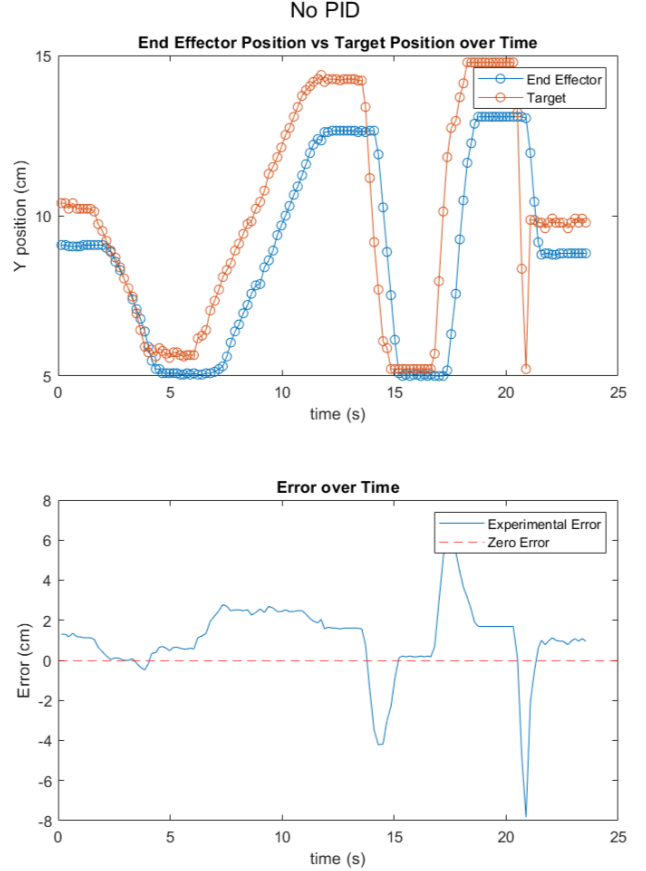


Fig. 5: End effector y -position (cm) versus time (s) for the controller with no PID tuning (top). The blue trace is the measured end effector position and the orange trace is the predicted target position. Individual data points are plotted with markers to reveal the ~ 7 – 15 Hz effective sampling rate. End effector position error $e_x(t)$ versus time for the final tuned controller (bottom). The dashed horizontal line at zero provides a reference for the ideal zero-error response.

Figure 5 shows that the controller doesn't ever converge over the trial period, and error is always present. **Figure 6** shows that increasing the proportional gain while holding the integral and derivative gains constant produced significant overshoot and oscillatory behavior during static periods of the trial. As a result, the system was unable to adequately settle at the desired position, preventing steady-state error convergence. **Figure 7** shows less transient tracking errors and less overshoot following abrupt target changes as closed-loop convergence is achieved. These deviations are most noticeable during the bounce shot at the end of the trial, causing the end effector to overcompensate for the large error. However, the controller remains stable and consistently converges to the desired position after each disturbance. The controller converges to the target within approximately 4.5 s following the bounce shot. **Figure 8** decreases the tracking errors and overshoot with abrupt changes even further. The time to converge following the bounce shot decreases to 2.5 s.

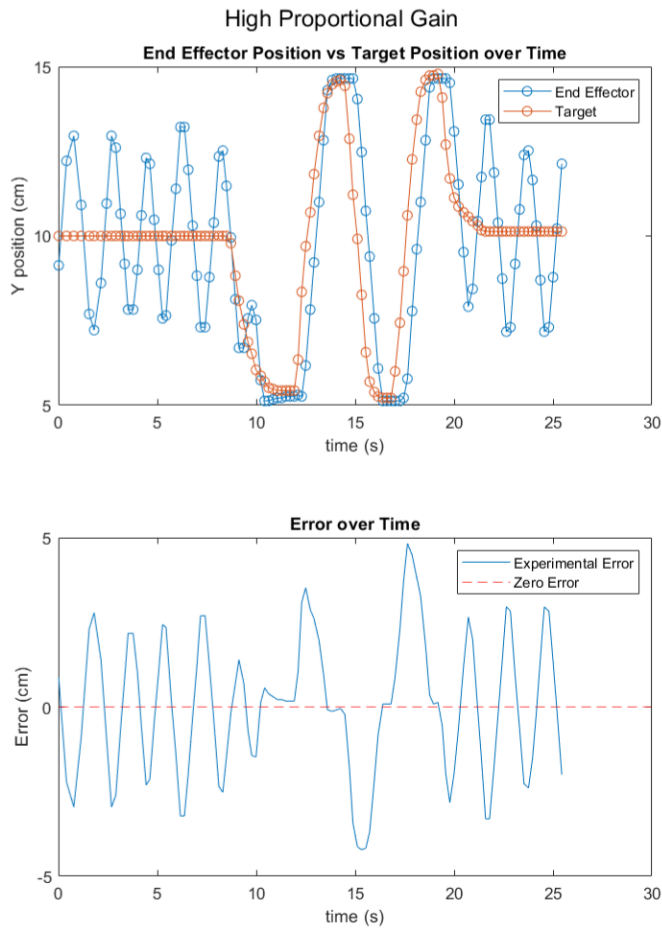


Fig. 6: End effector y -position (cm) versus time (s) for the controller with high proportional gain tuning (top). The blue trace is the measured end effector position and the orange trace is the predicted target position.

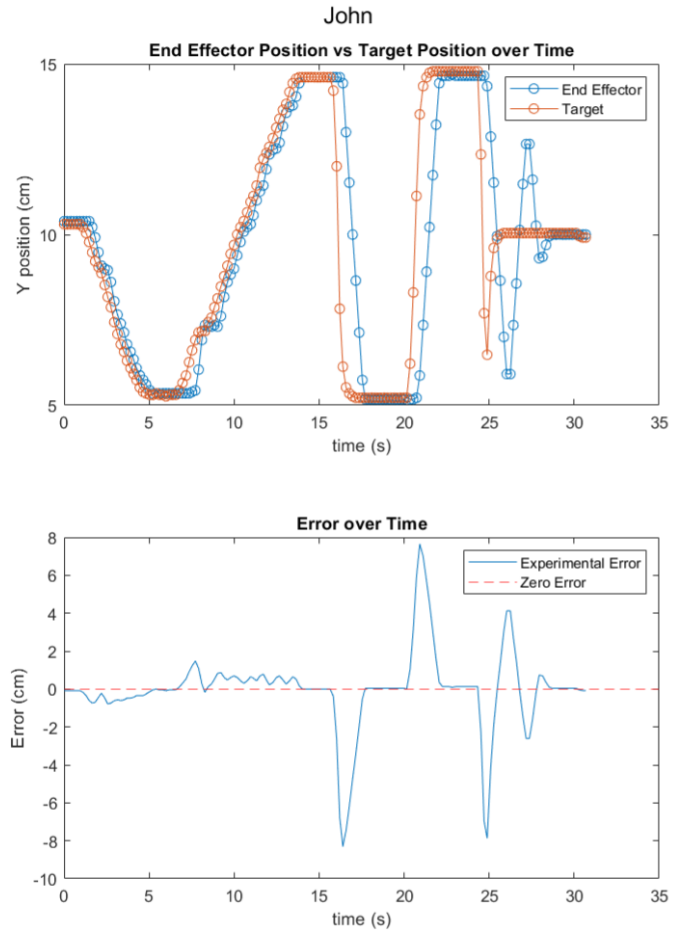


Fig. 7: End effector y -position (cm) versus time (s) for one of the team member's individual tuning ($K_P = 0.4$, $K_I = 0.009$, $K_D = 0.0325$) (top). The blue trace is the measured end effector position and the orange trace is the predicted target position. The controller tracks the target with low steady-state error during slow puck segments (~ 1.5 s– 13.5 s); large deviations correspond to abrupt target jumps caused by rapid puck direction changes (~ 16 s– 21.5 s). Bounce shot (~ 24.5 s– 30 s) shows large overshoot.

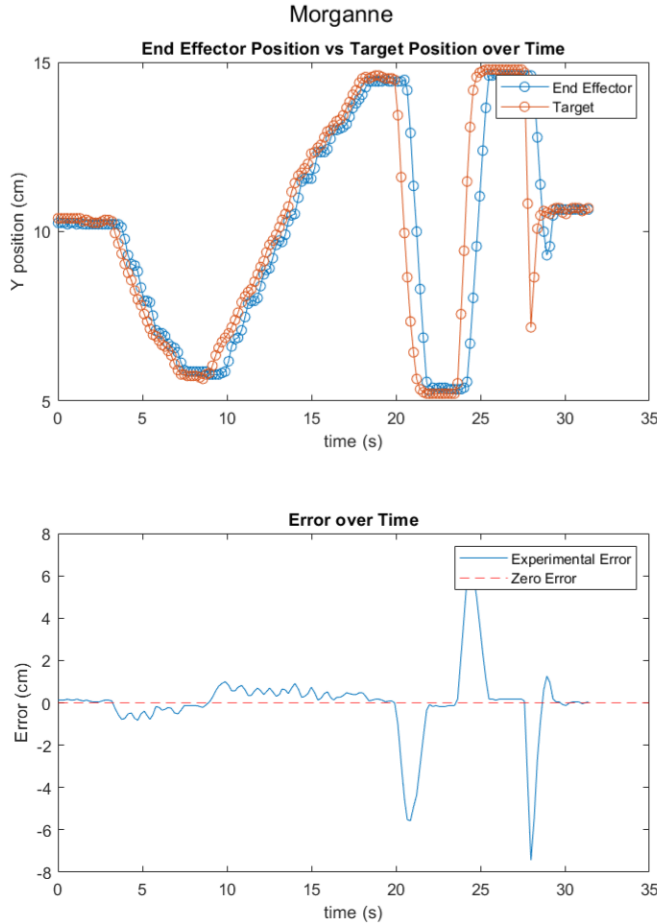


Fig. 8: End effector y -position (cm) versus time (s) for the final tuned controller ($K_P = 0.3$, $K_I = 0.004$, $K_D = 0.05$) (top). The blue trace is the measured end effector position and the orange trace is the predicted target position. The controller tracks the target with low steady-state error during slow puck segments (~ 3 s– 18 s); large deviations correspond to abrupt target jumps caused by rapid puck direction changes (~ 20 s– 25 s). Bounce shot (~ 26.5 s– 29.5 s) shows smaller overshoot than the other individual’s PID tuning values.

C. Performance Assessment

Table II summarizes the quantitative performance metrics for the final tuned controller evaluated using experimental puck-tracking trials. The selected controller gains of $K_P = 0.30$, $K_I = 0.004$, and $K_D = 0.05$ provided stable and accurate end-effector tracking throughout testing. The measured RMS tracking error was 1.54 cm, indicating that the controller maintained close agreement with the desired interception position during normal operation. The maximum observed tracking error was 6.94 cm and occurred during rapid target position changes, where delays in vision processing and actuator response introduced transient tracking lag. Compared to the no-PID configuration, which exhibited an RMS error of approximately 2.00 cm, the tuned PID controller improved tracking accuracy by approximately 23 percent. Additionally, the integral term effectively eliminated the steady-state offset

observed in proportional-only control, reducing the residual error to nearly zero during constant-position tracking.

TABLE II: Controller Performance Metrics (Final Tuned Gains: $K_P = 0.30$, $K_I = 0.004$, $K_D = 0.05$)

Metric	Value
RMS Tracking Error	1.54 cm
Peak Error	6.94 cm
Settling Time	~ 0.85 s
Overshoot	~ 0.8 cm
Steady-State Error (PID off)	~ 0.9 cm
Steady-State Error (PID on)	< 0.05 cm

D. Simulation Versus Experimental Results

No simulation environment was developed for this project. All performance data were obtained from hardware experiments on the physical air hockey table. The transfer function model as seen by equation (5) informed the qualitative expectations for the integrator-dominant plant response and motivated the use of a derivative term to reduce overshoot, but no closed-form simulation was used to pre-select gains. Gains were determined entirely through iterative hardware tuning guided by telemetry logs.

VI. DISCUSSION

A. Effect of Controller Gains

The effects of the PID gains were evaluated through experimental tuning while observing the robot’s ability to track commanded paddle positions and intercept incoming pucks. Increasing the proportional gain (K_P) improved the responsiveness of the system and reduced position error by commanding larger corrective actions. However, excessively large proportional gains caused oscillatory behavior and increased sensitivity to noise from the vision system. The integral gain (K_I) was introduced to eliminate steady-state error and improve convergence to the desired position. While larger integral gains reduced measurement error, excessive values resulted in overshoot and slower recovery following rapid trajectory changes. The derivative gain (K_D) provided damping by responding to the rate of change of the error signal, reducing oscillations and improving stability during aggressive maneuvers. However, high derivative gains amplified noise originating from the camera measurements and reduced controller smoothness.

Examining the transient response in **Figure 8**, the final gain set ($K_P = 0.3$, $K_I = 0.004$, $K_D = 0.05$) produced a slightly underdamped response with modest overshoot (~ 0.7 cm) at large step changes. The steady-state response was well-regulated: the integral term drove residual error within the ± 0.05 cm deadband for sustained tracking intervals, confirming the value of the integral action for eliminating offset due to friction and mechanical tolerance. For comparison, **Figure 7** shows the higher gain set ($K_P = 0.4$,

$K_I = 0.009$, $K_D = 0.0325$), which exhibited faster rise times but larger transient overshoot and slower recovery following the bounce shot (~ 4.5 s versus ~ 2.5 s). The final tuned gains were selected because the reduction in overshoot and settling time outweighed the minor decrease in initial response speed.

B. Controller Performance Evaluation

The final PID controller performed reliably as a defensive blocker against slow-to-moderate puck shots. The closed-loop system achieved consistent convergence to predicted intercept positions, with RMS tracking error of approximately 1.54 cm across puck-launch trials. Steady-state error was effectively eliminated by the integral term, reducing the residual offset from the ~ 0.9 cm observed under open-loop or proportional-only operation to within the software deadband. The saturation and clamping constraints prevented the arm from exceeding its physical table boundaries or causing over-torque events during aggressive commands.

Responsiveness degraded for fast puck shots. At the effective ~ 7 FPS operating rate (camera throughput reduced by motor communication overhead), the vision loop updated the target position on average only once every 140 ms, which is insufficient to track a puck crossing the table in under 400 ms. Under these conditions the robot frequently arrived at a stale target position rather than the true intercept point, resulting in missed interceptions.

C. Comparison of Controller Configurations

When designing and testing the controllers, two frameworks were considered: joint-space control and task-space control. The performance was assessed based on tracking error magnitude, convergence rate, interception reliability, and constraint enforcement.

In the joint-space implementation, the controller operated directly on motor positions. This approach made it difficult to integrate with the raw vision system output, as the camera inherently detects objects as pixel pairs within the task space. Consequently, this architecture required continuous, computationally intensive coordinate transformations between image coordinates and motor commands, leading to reduced interception accuracy. Testing confirmed lower positioning precision and longer settling times compared to the task-space approach.

Moving the control law into task space allowed the controller to operate directly on the paddle position using image coordinates, naturally aligning with the vision system's native pixel-coordinate output. Direct position comparisons simplified error calculation, resulting in improved tracking performance, more precise interception reliability, and a framework far easier to tune and integrate with the trajectory prediction model.

Operating without any structured PID controller resulted in large tracking errors, slow convergence, and occasional over-torque events. The PID implementation resolved these issues, improving position tracking accuracy, reducing steady-state error, enabling more reliable puck interception, and

successfully preventing the arm from exceeding its physical workspace boundaries.

The recommended controller configuration is the task-space decentralized PID with the final tuned gains ($K_P = 0.3$, $K_I = 0.004$, $K_D = 0.05$), as it provides the best balance of convergence speed, steady-state accuracy, and stability under the constraints imposed by the low-frame-rate vision system.

D. Capabilities and Limitations

The controller worked reliably for slower puck shots. It consistently moved the end effector toward predicted intercept positions and converged to within the deadband during sustained tracking intervals.

The main limitation was the camera frame rate. A ~ 15 FPS nominal limit was reduced further to ~ 7 FPS when motor commands were being transmitted concurrently, because the Raspberry Pi 4B lacked sufficient headroom to run the camera and serial communication simultaneously at full speed. Fast shots produced stale puck measurements that caused delayed end effector motion, particularly when the human opponent faked a shot and changed direction late. These misdirections, combined with the low camera frame rate, delayed trajectory correction and moved the end effector toward the wrong target. Edge shots that drove the predicted intercept near the workspace limits also exploited increased belt tension, slowing the end effector in regions where faster motion was needed most.

E. Challenges Encountered

HSV color thresholding was sensitive to uneven lighting across the table surface. As lighting varied with object position, the color values of the puck, paddle, and end effector shifted between trials and occasionally required manual recalibration of HSV bounds. Initially, the puck and opponent paddle were both bright red, making them visually indistinguishable in the segmented mask. This was resolved by replacing the top surface of one paddle with a different-colored cap, creating a green-topped "super paddle" that provided sufficient contrast for reliable segmentation. The end effector was initially printed in white PLA, which blended into the table surface; blue tape was applied to create a distinct color signature.

Frame rate further decreased as software complexity grew, due to the Raspberry Pi's limited processing headroom. Motor communication also required a reinstallation of Python 3.11 and switching to command-terminal execution to resolve SDK compatibility issues.

Near the workspace edges, the pulley design increased belt tension and sometimes caused over-torque or belt skipping. The added resistance slowed the end effector at the periphery of the workspace, making edge shots a viable human strategy for defeating the robot. Hard boundary limits in both task space and the defensive position logic were added to prevent the arm from entering these problematic zones.

Implementing a functional control loop was the primary software challenge once hardware was assembled. A clear

steady-state error of ~ 3 cm was visible from the camera feed before the integral term was activated. Tuning was iterative: K_P was stepped down logarithmically from 10 through 1 to 0.1, settling between 0.3 and 0.4. Gains too high caused wild oscillation; gains too low produced stuttering motion that never settled. Once a suitable K_P was established, K_I and K_D tuning became more straightforward.

A controller that could be defeated by certain human strategies provides a concrete way to probe the system's limits. An opponent who (1) quickly reverses the puck direction after the robot has already begun moving to one side, (2) aims shots toward the workspace edges where belt tension slows the end effector, or (3) launches the puck at high speed so that the 140 ms update interval is too slow to react, can reliably score against the current system. These strategies directly exploit the low camera frame rate, communication latency, and edge-zone mechanical compliance.

VII. FUTURE DIRECTIONS

Future work should add an attack function for offensive play. This could be achieved by adding a second control phase after interception. Once the end effector reaches the intercept point, the controller would command a follow-through motion in a selected return direction, allowing the system to return shots instead of only blocking them.

Instead of HSV color thresholding, using AprilTags would improve tracking under changing lighting and allow camera pose estimation during operation, reducing dependence on a fixed camera location. Implementing a higher-FPS camera would reduce the delay between puck measurements and improve system response to fast shots and late direction changes.

Mechanical updates would focus on reducing carriage backlash by improving the stiffness of the robotic links. Belt tensioners and improved pulley alignment would achieve more consistent belt tension, enabling faster and more repeatable direction changes near the workspace limits.

Feedforward control should be added to improve response time and interception success. A feedforward term would command motion toward the predicted intercept point before large tracking error develops, reducing lag when the target changes quickly. The remaining position error would then be corrected through PID feedback. With a higher-frame-rate camera, reliable velocity estimates would make this feedforward term practical, unlike in the current system where frame-to-frame velocity measurements are too noisy to use.

VIII. CONCLUSION

This project developed a 2-DOF air hockey robot for puck interception using a decentralized PID controller operating in task (pixel) space. The system detected the puck and opponent paddle, predicted the puck trajectory including wall bounces, and commanded an intercept position. The PID controller enabled reliable defensive blocking during slower puck shots, reducing steady-state error from ~ 3 cm to within the software deadband and achieving an RMS tracking error of ~ 1.54 cm.

Performance degraded for fast shots due to the ~ 7 FPS effective vision update rate and Pi-to-motor communication latency. Task-space control outperformed joint-space control by naturally aligning with the vision pipeline and eliminating the need for repeated coordinate transformations. Future work should target faster sensing, AprilTag-based tracking, feed-forward control, and improved mechanical stiffness to enable reliable interception of high-speed shots and offensive play.

ACKNOWLEDGMENT

The authors would like to thank the creator Zeroshot (via Instructables) for the open-source air hockey robot design [2], [3] which served as the mechanical basis for this project. The authors also utilized large language model (LLM) AI tools to assist in code debugging and interfacing with the physical electronics.

REFERENCES

- [1] Z. Sun, Y. Zhou, C. Zhang, X. Zhang, S. Li, and H. Yu, "Decentralized Robust Exact Tracking Control for 2-DOF Planar Robot Manipulator," in *Proc. 2018 3rd International Conference on Advanced Robotics and Mechatronics (ICARM)*, Singapore, 2018, pp. 655–660, doi: 10.1109/ICARM.2018.8610815.
- [2] Zeroshot, "Air Hockey Robot," *Instructables*, 2024. [Online]. Available: <https://www.instructables.com/Air-Hockey-Robot/>
- [3] Zeroshot, "Air Hockey Robot," *YouTube*, 2024. [Online]. Available: <https://www.youtube.com/watch?v=VZdKkK-IPW4>

APPENDIX A

TEAM MEMBER CONTRIBUTIONS

All team members contributed to the controller design process, including PID gain tuning through iterative hardware trials. Individual contributions are described below.

Spencer Kelly served as team lead and was the primary developer of the vision subsystem. He implemented the full HSV color thresholding pipeline, contour detection, centroid extraction, and the HSV tuner calibration tool. He also led software integration of the closed-loop system and coordinated technical decisions across subsystems.

John Meshinsky was responsible for motor testing and hardware tuning. He characterized Dynamixel motor behavior, verified axis directions and workspace limits, and iteratively adjusted motor profile parameters to achieve reliable actuation throughout the table workspace.

Hannah Yared led the error analysis and controller design work. She analyzed telemetry logs to assess tracking and error performance, produced performance plots, and contributed to the mathematical formulation of the control architecture and system modeling sections.

Morganne Malloy and **Tony Zong** were primarily responsible for the design, manufacturing, and physical assembly of the robot. This included fabricating and assembling the belt-driven carriage, mounting the linear rails and pulleys, 3D printing the custom end effector defender paddle, and preparing the air hockey table for testing.

APPENDIX B BILL OF MATERIALS

Item	Qty	Cost	Vendor
Air Hockey Table	1	\$21.95	Home Depot
RP Camera (Vision)	1	\$15.35	Amazon
Rods & Batteries	1	\$17.97	Amazon
Belt and Pulleys	1	\$24.98	Amazon
Rods	1	\$10.92	Amazon
Bearings	1	\$13.16	Amazon
Total		\$104.33	-

TABLE III: Bill of Materials

APPENDIX C CUSTOM CAD DESIGNS

CAD models for the custom 3D-printed end effector defender paddle and camera mounting bracket are included in the shared project Google Drive folder.

APPENDIX D CUSTOM CODE

The full project codebase is available in the shared project Google Drive folder. Manipulator code was written in Python, and MATLAB was used for data processing.

APPENDIX E ELECTRICAL AND WIRING DIAGRAMS

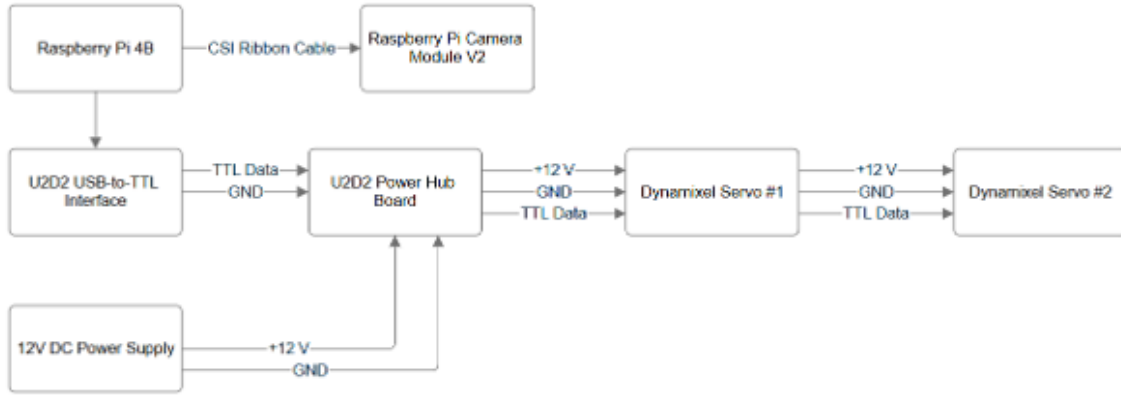


Fig. 9: System wiring diagram.

The electronics consist of a Raspberry Pi 4B connected to two Dynamixel servo motors via a U2D2 USB-to-TTL interface board, powered by a 12 V supply. The Raspberry Pi Camera Module V2 connects via the CSI ribbon cable. Wiring diagrams are included in the shared project Google Drive folder.

APPENDIX F SUPPLEMENTAL FIGURES AND TABLES

A video demonstrating the robot performing defensive blocking against a human opponent is available at the link below. The video shows the vision overlay (puck centroid, paddle centroid, end effector centroid, and predicted trajectory line) alongside real-time motor motion.

https://youtube.com/shorts/g_Lt8tYEROQ?feature=share